A Transparent Financial Risk Factor Library

Christoph Frey Christoph Scheuch Stefan Voigt
Patrick Weiss

We introduce `tidyfinance`, an R package on CRAN for the core data workflow in empirical asset pricing: downloading stock returns and financial statements, constructing sorting variables, forming quantile portfolios, and computing long-short factor returns. A unified `download_data()` interface covers WRDS (CRSP, Compustat), Ken French's Data Library, FRED, and further sources. Modular functions handle fiscal-year lagging, breakpoint computation, and portfolio aggregation; every construction choice has a documented default and can be varied independently, making pipelines transparent and reproducible. We illustrate the `tidyfinance` workflow with a risk factor library covering 50 sorting variables across 187008 precomputed specifications. The library is accessible via an open-source API and an interactive dashboard at factors.tidy-finance.org.

Introduction: Constructing financial risk factors

> **Note**
>
> Core contribution of the paper: `tidyfinance`, a modular, open-source library for reproducible workflows in empirical asset pricing. The relevant steps are a unified data access interface, modular functions for data processing, especially lagging, filtering and implementing portfolio sorts. A secondary contribution is the open-source factor library covering 50 sorting variables across 187008 specifications. The library is available via an API and an interactive dashboard at factors.tidy-finance.org.
>
> - Python is not primary focus of the paper. We mention `py-tidyfinance` in the very end of the introduction to highlight access to the factor library and once when we show how to use `download_data`.

Factor returns drive empirical asset pricing tests, portfolio allocation, and performance attribution. Every series, however, embeds dozens of methodological choices: how breakpoints are set, whether portfolios are equal- or value-weighted, how delistings are handled, and which

fiscal-year lag is applied. These choices are rarely standardized and almost never published with code. Researchers who reproduce or extend a factor series must reconstruct each pipeline from scratch, with no guarantee their choices match those of the original authors.

No widely adopted, transparent source standardizes factor construction. Current practice relies on informal publication of factor series without code or documented workflows, limiting reproducibility, comparability, and extension. Existing resources illustrate the gap: the French Data Library (Eugene F. Fama and French 2023) provides canonical factors but no code; the Open Source Asset Pricing Project (Chen and Zimmermann 2022) focuses on replication; the global factor database (Jensen, Kelly, and Pedersen 2023) targets Python only. No unified, open-source alternative exists.

We introduce `tidyfinance`, a modular framework (R on CRAN, Python on PyPI) for reproducible factor construction. The library covers 50 sorting variables and 187008 total specifications. We document and test every design choice (breakpoints, weighting, rebalancing, sample inclusion) and expose each as a configurable parameter.

The package implements an end-to-end pipeline: data acquisition from the Wharton Research Data Services (WRDS), Compustat, and other common providers; automated variable construction with standard lagging and cleaning routines; and portfolio aggregation with configurable sorting. A programmatic application programming interface (API) provides access to precomputed factor series with versioning and batch retrieval. Researchers using python access the same data download functions and factor library via `py-tidyfinance`. An interactive dashboard at factors.tidy-finance.org makes the library accessible without code. All software and workflows are publicly available to enable replication, sensitivity analysis, and extension.

Hou, Xue, and Zhang (2020) replicate 452 cross-sectional anomalies and find that most fail to survive out-of-sample, attributing failures in part to undocumented construction variation across studies. Chen and Zimmermann (2022) respond by publishing code for 212 anomaly portfolios, but their library targets a fixed, documented set rather than a reconfigurable pipeline. Jensen, Kelly, and Pedersen (2023) provide a global factor database without an open construction workflow. `tidyfinance` fills the gap all three leave open: every construction decision is explicit, configurable, and version-controlled, so researchers can trace and vary any assumption systematically.

Among R packages for quantitative finance, tidyquant (Dancho and Vaughan 2026), PerformanceAnalytics (Peterson and Carl 2026), and quantmod (Ryan and Ulrich 2025) support time-series retrieval and portfolio performance analytics. None begins at raw accounting and return data, applies the cleaning and lagging conventions factor construction requires, or exposes construction choices as configurable parameters. `tidyfinance` provides this upstream pipeline with a consistent API across both languages. Hence, it enables researchers to access standardized factor series without rebuilding data workflows in each environment.

Package overview

> **Note**
>
> This section introduces the design choices and workflow of `tidyfinance`.
>
> - Installation
> - Brief description of the workflow (data acquisition, data processing, portfolio sorting) in general. Illustration how `tidyfinance` offers support along these dimensions with a subsection on each part.
> - Data access: Show which data sources are available via `download_data()`, with examples of how to call the function for each source. We mention `py-tidyfinance` here once to highlight that the same function is available in Python.
> - Also highlight the standard cleaning conventions applied to CRSP, Compustat, TRACE data.
> - Define what `date` means in `tidyfinance` conventions (i.e. end of month for monthly data, fiscal year-end for annual data, etc.)
> - Data processing: Description and illustration of lagging with `add_lagged_columns()`, showing how the function implements the Fama-French convention for fiscal-year lagging.
> - Data processing: Brief mention and explanation of other useful helper functions (`winsorize`, `trim`, `estimate_model`, `compute_rolling_value`)
> - Portfolio sorting: Description and illustration of the methodology as such as well as the relevant functions (`compute_breakpoints()`, `assign_portfolio()`, `compute_portfolio_returns()`). Describe the use of `implement_portfolio_sort()` as a wrapper that implements the full pipeline from sorting variable to portfolio returns in one step.

Installation and setup

You can install the released version of `tidyfinance` from CRAN via:

```
install.packages("tidyfinance")
```

To install the development version of `tidyfinance` from GitHub, first install the pak package if needed, then install the development version:

```
pak::pak("tidy-finance/r-tidyfinance")
```

Design and workflow

`tidyfinance` organises the empirical asset pricing workflow into three stages: data acquisition, variable construction, and portfolio sorting. Each stage involves dozens of small decisions, such as universe filters, cleaning rules, rebalancing conventions, and breakpoint definitions. These decisions are rarely documented uniformly across studies, making replication difficult. Figure **??** shows the pipeline and the package functions at each stage. The functions expose documented defaults that follow established conventions and configurable parameters where reasonable researchers might disagree. In this way, researchers can adopt the full pipeline or call individual functions within their own workflows.[1]

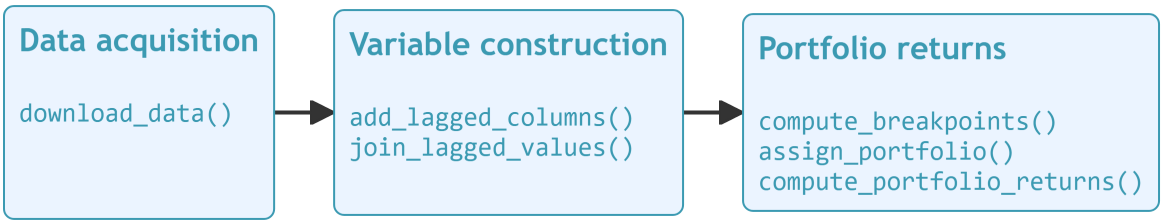


Figure 1: Research workflow for factor construction with `tidyfinance`, illustrating the three-stage pipeline and the package functions involved at each step.

Data access

`tidyfinance` provides a single interface, `download_data()`, to 319 dataset types: equity returns, fixed-income transactions, macroeconomic predictors, index constituents, and more. Table **??** lists all available sources. `list_supported_types()` and `list_supported_indexes()` return the full catalogue programmatically.

Table 1: Data sources available via `download_data()` in `tidyfinance`.

Source	Description	Example
Public data sources		
Ken French's Data Library	Fama-French 169+ factor models, portfolios, industry classifications	`download_data( "famafrench", "Fama/French 5 Factors")`

[1] A companion Python package `tidyfinance` provides the same interface with identical function signatures and dataset identifiers, so analyses can be ported between the two languages without changes to the data layer.

Source	Description	Example
Global Q	Hou-Xue-Zhang Q5 factor model (market, size, investment, ROE) at multiple frequencies	`download_data("globalq", "factors_q5_monthly")`
Goyal-Welch Predictors	Macroeconomic return predictors (DP, EP, BM, spreads, etc.)	`download_data("macro_predictors", "monthly")`
FRED	St. Louis Fed economic time series (GDP, CPI, etc.)	`download_data("fred", series = "GDP")`
Yahoo Finance	Daily OHLCV with adjusted close for any ticker	`download_data("stock_prices", symbols = "AAPL")`
Index Constituents	Current members of 21 major indexes (S&P 500, Russell, DAX, MSCI World, etc.)	`download_data("constituents", index = "DAX")`
Open Source Asset Pricing	213+ monthly asset pricing signals and anomalies	`download_data("osap")`
WRDS (subscription required)		
CRSP	Stock returns, market cap, exchange, industry	`download_data("wrds", "crsp_monthly")`
Compustat	Annual and quarterly financial statements	`download_data("wrds", "compustat_annual")`
CCM Links	CRSP-Compustat linking table (permno to gvkey)	`download_data("wrds", "ccm_links")`
FISD	Mergent Fixed Income Securities Database	`download_data("wrds", "fisd")`
TRACE	Corporate bond transactions (Dick-Nielsen cleaned)	`download_data("wrds", "trace_enhanced", cusips = "655844CJ5")`

`download_data()` applies standard cleaning conventions automatically. For CRSP monthly data (version 2), the function restricts the universe to common equity securities traded on NYSE, AMEX, or NASDAQ. It computes market capitalisation as shares outstanding times price, constructs a one-month lag for portfolio weights, and maps exchange codes and SIC codes to readable labels. Excess returns are raw returns minus the Fama-French risk-free rate; observations with missing returns or market capitalisation are dropped. CRSP version 2 incorporates delisting returns directly, eliminating the need for a separate adjustment.

For Compustat annual data, `download_data()` restricts to industrial-format consolidated

statements and retains only the most recent observation per firm-year. It constructs book equity through a coalesce hierarchy: stockholders' equity, then common equity plus preferred stock, then total assets minus total liabilities. The function adjusts for deferred taxes and investment tax credits and subtracts preferred stock.

All datasets returned by `download_data()` use a single `date` column that follows the same rules across frequencies. Daily observations return the trading date. Monthly observations are dated to the last calendar day of the reference month, regardless of whether that day was a trading day. Quarterly and annual observations take the fiscal period-end date as reported by the original source; for Compustat, this is the firm's fiscal year-end (`datadate`), which can differ from December across firms and across years for the same firm. Macroeconomic series from FRED keep their original release dates. A simple equality join on `date` therefore aligns observations of the same frequency, while cross-frequency joins, such as merging monthly returns with annual fundamentals, require the lagging logic described in the next subsection.

Variable construction

Most predictive variables in asset pricing studies are based on lagged information so that portfolio formation in month t should always use only data observable at the end of month $t-1$. `add_lagged_columns()` automates this. For each row in the data, it looks back over a configurable window and attaches the most recent observed value of the chosen columns. By default (`ff_adjustment = FALSE`), the window is bounded by `lag` (the minimum delay between observation and use) and `max_lag` (the maximum delay before a stale value is treated as missing). Within this window, the function picks the most recent non-missing value; if no observation falls in the window, the lagged column takes `NA`.

A second mode, enabled by `ff_adjustment = TRUE`, implements the Fama-French convention (Eugene F. Fama and French 1993). Accounting variables for fiscal year $t-1$ are assumed to become public information by June of year t and are then used for portfolio formation from July of t through June of $t+1$. This produces step-function lagged values that update once per year at the June/July boundary, regardless of the underlying fiscal year-end. Figure **??** contrasts the two mechanisms. The upper panel shows a standard look-back window controlled by `lag` and `max_lag`. The lower panel implements the Fama-French convention: fiscal-year data from year $t-1$ becomes available only from July of year t, so all months from July through June share the same lagged value locked in at the previous rebalancing. In both cases, `add_lagged_columns()` selects the most recent observed value within the window.

`join_lagged_values()` is a related helper that merges lagged values from one data frame into another using the same windowing logic. It is useful when the target panel and the source panel are at different frequencies, for example when joining monthly accounting data into a quarterly target panel. The function takes the same `ff_adjustment` argument as `add_lagged_columns()`, so the Fama-French convention can be applied at the join step rather than as a separate transformation.

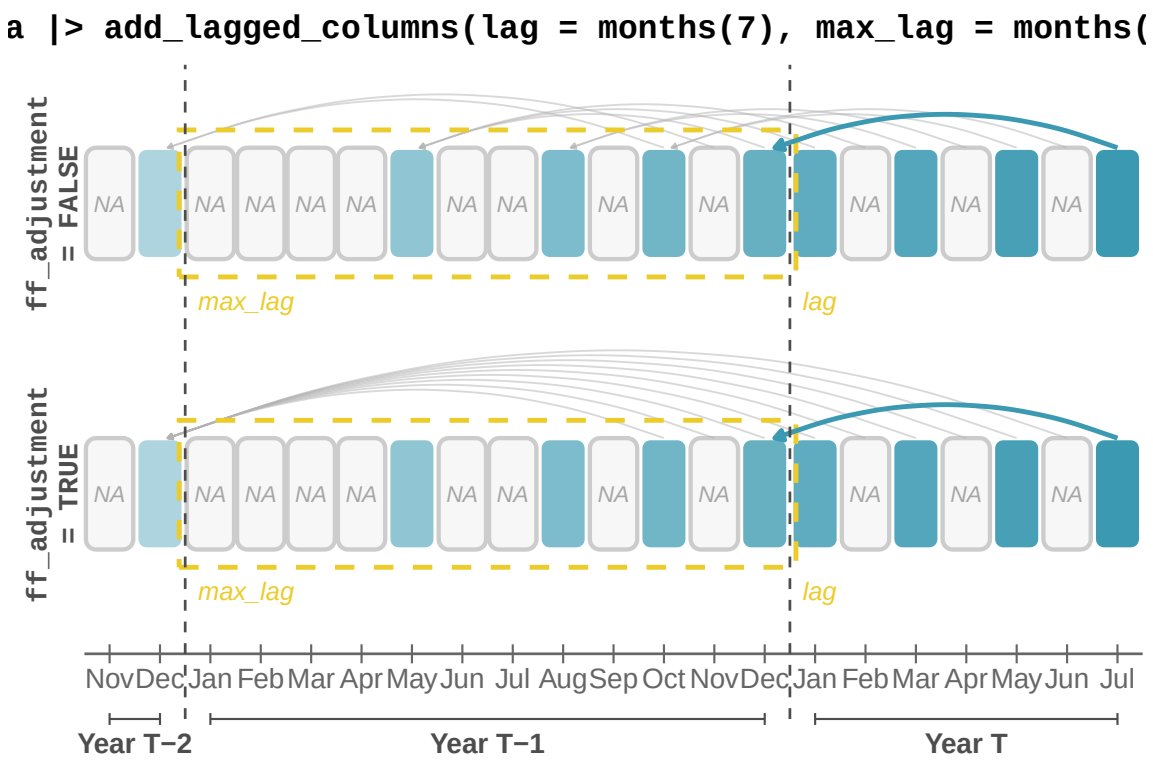


Figure 2: Two lagging mechanisms in `add_lagged_columns()`. Each row represents a data series with observed (filled) and missing (hollow) cells. Dashed rectangles mark the search window; arrows show which value gets matched to each target month.

A handful of additional helpers cover routine operations that recur across empirical studies. `winsorize()` and `trim()` handle outliers in cross-sectional variables: the former replaces values beyond user-specified quantiles with the quantile values themselves, while the latter sets them to `NA`. `estimate_model()` fits a linear model from a formula string and returns the coefficients as a tidy one-row data frame, which composes naturally with `group_by()` for grouped or rolling regressions such as time-series factor exposures. `compute_rolling_value()` applies an arbitrary aggregation function over a rolling window defined in calendar time rather than row counts, accommodating irregular observation frequencies and missing periods.

Portfolio sorting

Portfolio sorts are among the most widely used tools in empirical asset pricing (Bali, Engle, and Murray 2016). The procedure groups stocks each period into portfolios based on a sorting variable, such as size, beta, book-to-market, or momentum, and then compares return distributions across the resulting portfolios. The long-short return between the extreme portfolios is interpreted as a factor-mimicking strategy and serves as the basis for tests of whether the sorting variable forecasts future returns. We expose the procedure through four composable functions.

`compute_breakpoints()` takes a cross-section and a sorting variable and returns the numeric cutoffs that define the portfolio boundaries. The user supplies either a number of portfolios (for example, five for quintiles) or an explicit vector of percentiles, but not both. A `breakpoints_exchanges` argument restricts the universe used to compute breakpoints. The standard convention is NYSE-only, which prevents the large cross-section of small NASDAQ firms from dominating the lower bins. A related `breakpoints_min_size_threshold` argument excludes stocks below a market-cap quantile from the breakpoint computation, which is useful for size-related sorts where microcaps would otherwise distort the cutoffs. A `smooth_bunching` flag handles cases where the sorting variable clusters at the edges by pinning extreme values to the boundary portfolios and recomputing breakpoints on the remainder.

`assign_portfolio()` maps each observation to a portfolio integer using the breakpoints. The breakpoint computation is delegated to a user-supplied function (defaulting to `compute_breakpoints()`), so custom rules such as dollar-based cutoffs or industry-adjusted breakpoints can be plugged in without rewriting the assignment logic. In practice, `assign_portfolio()` is called separately for each rebalancing date, so that portfolios are reformed each period rather than fixed once for the whole sample.

`compute_portfolio_returns()` ties the lower-level pieces together into a single call. Given a panel of stock data, one or two sorting variables, a sorting method, and `breakpoint_options`, it handles rebalancing, breakpoint computation, portfolio assignment, and return aggregation. It then returns a complete panel of portfolio returns. For each portfolio-date the function reports three return measures: equal-weighted, value-weighted using `mktcap_lag` as the weight, and capped value-weighted. The capped version pulls weights above the `cap_weight` percentile

(default 0.8) back to that cap, which prevents a small number of megacap firms from dominating the portfolio. A `min_portfolio_size` argument sets the minimum number of constituents below which the corresponding return is set to `NA`. An optional `rebalancing_month` argument switches from period-by-period rebalancing to annual rebalancing in the specified month. Portfolios formed in that month are then held constant for the following twelve months.

The function supports three sorting schemes, illustrated in Figure **??**. A *univariate* sort uses a single variable. An *independent bivariate* sort computes breakpoints on each variable separately and intersects the resulting bins to produce a two-way grid. A *dependent bivariate* sort first partitions the cross-section by the secondary (control) variable and then re-sorts within each bucket on the main variable, which is the appropriate choice when the second variable is treated as a control rather than as an independent dimension of variation. For bivariate sorts, returns are averaged over the secondary variable so that only the main-variable portfolios are returned.

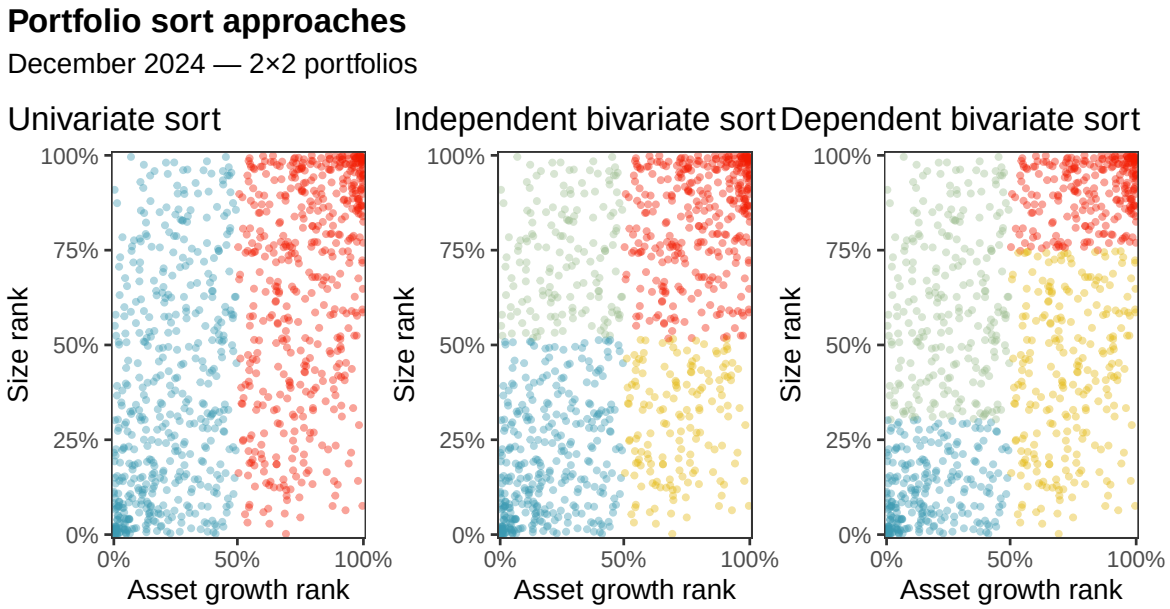


Figure 3: Portfolio sort approaches illustrated for a single month (December 2024). Each panel plots stock-month observations assigned to `r .QuartoInlineRender(n_portfolio)`×`r .QuartoInlineRender(n_portfolio)` portfolios based on asset growth rank (x-axis) and size rank (y-axis), both expressed as percentiles. Colors indicate portfolio cell assignment. The univariate sort (left) assigns portfolios on asset growth alone; the independent bivariate sort (center) assigns each dimension independently; the dependent bivariate sort (right) sorts on size first, then on asset growth within size groups.

`compute_long_short_returns()` is a thin convenience layer that takes the output of

`compute_portfolio_returns()` and computes the spread between the top and bottom portfolios. A `direction` argument controls the sign and defaults to `"top_minus_bottom"`. The function returns one long-short series per return measure available in the input, so the equal-weighted, value-weighted, and capped value-weighted spreads are produced in a single call. This is the long-short factor-mimicking return that is usually the object of interest in the asset-pricing literature.

Vignette: A case study in factor construction

> **Note**
>
> This section shows the workflow from scratch without excessive documentation by going through data aggregation, variable construction for asset growth, lagging and sorting into portfolios. The purpose is to illustrate the workflow and the relevant functions in a concrete example. We do not show every step of the data cleaning process for brevity.
>
> - Compute asset growth risk factor time series once with defaults (describe the defaults) using `implement_portfolio_sort()`.
> - use the `filter_options`, `breakpoint_options`, and other arguments to streamline the process for an *exotic* example.
> - Illustrate "standard" summary stats for one specification of the factor or use fama-mac beth functionality for some standard testing.

We walk through the construction of the asset growth factor from raw CRSP and Compustat data to long-short portfolio returns, implementing each stage of Figure **??** in turn. Each step uses a `tidyfinance` function and generalises directly to any sorting variable in the factor library.[2]

Once installed, load `tidyfinance` and a minimal set of packages for data manipulation and visualisation.

```
library("tidyfinance")
```

```
library("dplyr")
library("tidyr")
library("lubridate")
```

[2]Installation of the `tidyfinance` package is described in the appendix in Section **??** Installation and setup.

Data download

Before the first WRDS download, call `set_wrds_credentials()` to store WRDS credentials securely in `.Renviron`; subsequent `download_data()` calls establish the database connection automatically.

```
date_start <- as.Date("1960-01-01")
date_end <- as.Date("2024-12-31")
```

First, we download monthly stock returns from CRSP. Setting `add_ccm_links = TRUE` appends CRSP-Compustat linking identifiers needed to merge with financial statement data in the next step.

```
crsp_monthly <- download_data(
  domain = "wrds",
  dataset = "crsp_monthly",
  version = "v2",
  start_date = date_start,
  end_date = date_end,
  add_ccm_links = TRUE
)
```

The dataset contains 3451594 stock-month observations for 26263 unique stocks, covering the period from 1960-01-01 to 2024-12-01.

Next, we download annual financial statements from Compustat.

```
compustat_annual <- download_data(
  domain = "wrds",
  dataset = "compustat_annual",
  start_date = date_start,
  end_date = date_end,
  additional_columns = c("at", "ib"),
  only_us = TRUE
)
```

Annual Compustat data contains 506778 firm-year observations for 39587 unique firms, covering the period from 1960-01-01 to 2024-12-01. The `additional_columns` argument retrieves total assets (`at`) and income before extraordinary items (`ib`), needed for computing asset growth and applying an earnings filter in later steps. Setting `only_us = TRUE` restricts the dataset to USD-denominated observations, ensuring consistency with CRSP.

Data cleaning and variable construction

We illustrate data cleaning and variable construction using asset growth (AG) as our running example. Asset growth is the year-over-year percentage change in total assets (Cooper, Gulen, and Schill 2008),

$$AG_t = (AT_t - AT_{t-1})/AT_{t-1}.$$

The `add_lagged_columns()` function from `tidyfinance` handles temporal alignment by creating a lagged version of total assets grouped by firm identifier (`gvkey`) with a one-year lag. Non-finite values (arising when lagged total assets are zero or missing) become `NA`.[3]

```
sorting_variable_data <- compustat_annual |>
  add_lagged_columns(
    "at",
    lag = years(1),
    by = "gvkey",
  ) |>
  mutate(
    asset_growth = (at - at_lag) / at_lag,
    asset_growth = if_else(is.finite(asset_growth), asset_growth, NA_real_)
  ) |>
  select(gvkey, date, asset_growth)
```

`sorting_variable_data` contains one observation per firm-year, the natural unit for cross-sectional analysis of annual accounting variables. The package provides `filter_out()` as a convenience wrapper that removes rows matching a logical predicate, and `winsorize()` to clip numeric vectors at specified quantile bounds. Asset growth is heavily right-skewed: a small number of firms with dramatic balance-sheet expansions generate extreme positive values. We winsorize at the 1st and 99th percentiles before computing summary statistics. Table **??** reports the pooled cross-sectional summary; Table **??** shows how those moments vary across calendar years.

[3] For the sake of simplicity we do not show every step of the data cleaning process in the paper. For instance, we apply an earnings filter to exclude firms with negative or zero income before extraordinary items (`ib`), which is a common practice in the literature to mitigate the influence of financial distress on asset growth portfolios. This filter is implemented in the code but not shown in the paper for brevity. We also apply an age filter to exclude firms younger than two years, which is common in the literature to ensure that the sorting variable is based on stable financial information. This filter is also implemented in the code but not shown in the paper for brevity. The factor library API provides preprocessed panels with all these filters applied, so users can access the final sorting variables without having to implement these steps themselves.

Table 2: Cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). The table reports the number of firm-year observations (N), the mean, standard deviation (SD), and selected percentiles of the annual asset growth distribution computed from `sorting_variable_data`.

N	Mean	SD	P10	P25	Median	P75	P90	
389123	0.238	0.858	-0.191	-0.034	0.064	0.207	0.588	

Table 3: Time-series averages of annual cross-sectional summary statistics for asset growth (winsorized at the 1st and 99th percentiles). For each calendar year, cross-sectional mean, standard deviation, and median are computed from `sorting_variable_data`; the table reports the time-series mean, minimum, and maximum of those annual statistics.

Years	Avg N	Mean (avg)	Mean (min)	Mean (max)	SD (avg)	Median (avg)	Median (min)	Median (max)
64	6080	0.242	0.048	0.889	0.847	0.067	0.001	0.132

The next step merges annual sorting variables onto the monthly CRSP stock panel using `join_lagged_values()`. The function aligns fiscal-year-end data with monthly observations through a configurable lag window. Setting `ff_adjustment = TRUE` implements the Fama-French convention (Eugene F. Fama and French 1993), aligning fiscal-year data from year t with portfolio formation in July of year $t+1$ through a 12-to-18-month look-back window. The same lagging process generalizes to all sorting variables in the full factor library. The factor library API provides precomputed lagged panels for all variables.

With both lagging mechanisms documented, we apply `join_lagged_values()` to merge the sorting variable onto the monthly return panel. Figure **??** shows the 33rd, 50th, and 66th percentiles of asset growth over time in the resulting panel; pronounced spikes during economic expansions and contractions highlight the cyclical nature of corporate investment that portfolio sorts aim to exploit.

```
sorting_data <- crsp_monthly |>
  join_lagged_values(
    sorting_variable_data,
    "gvkey",
    min_lag = months(7),
    max_lag = months(18),
    ff_adjustment = TRUE
  )
```

```r
sorting_data <- sorting_data |>
  select(date, permno, exchange, ret_excess, mktcap_lag, asset_growth) |>
  filter(date > date_start + months(12) + months(18))
```

The filter removes the initial year used to compute the growth rate and an additional 18 months to avoid mechanically introduced gaps from the lagging step, leaving a panel of stock-month observations with excess returns, exchange membership, lagged market capitalization, and the lagged sorting variable.

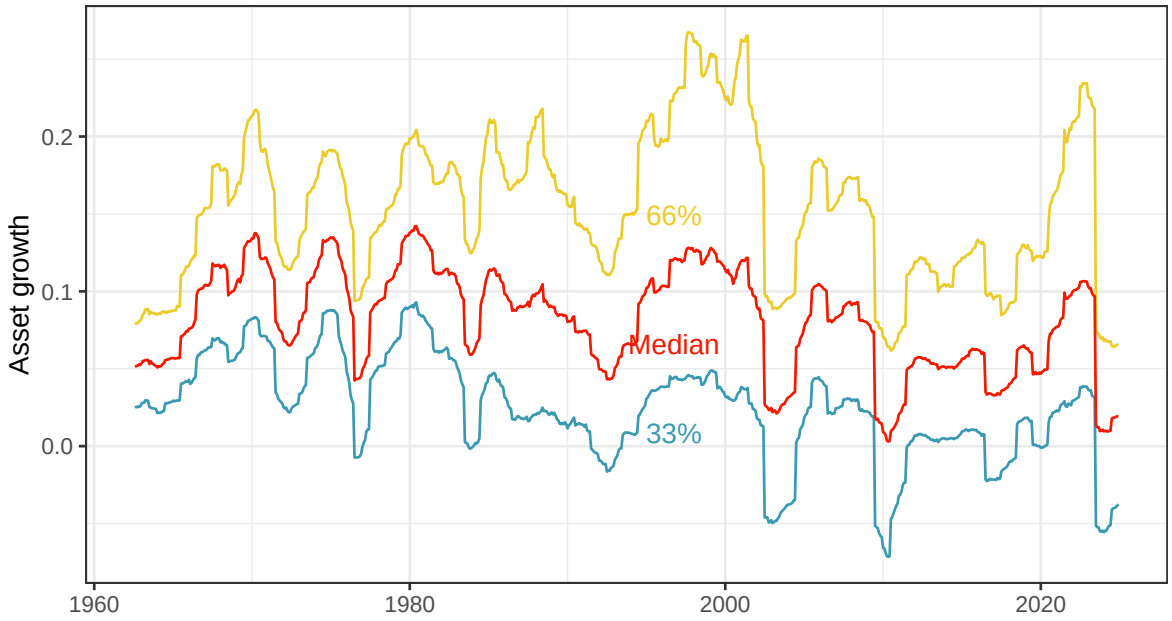


Figure 4: Distribution of asset growth over time. Lines show the 33rd, 50th, and 66th percentiles of the cross-sectional distribution of asset growth in `sorting_data`, the firm-month panel used as input for portfolio construction. Asset growth is expressed as a proportion (e.g., 0.2 = 20% annual growth in total assets) and covers the full sample period of the merged CRSP-Compustat dataset.

With the sorting variable constructed and merged onto the monthly stock panel, we now turn to portfolio sorts, the mechanism that translates firm-level characteristics into tradable factor returns.

Portfolio sorts

In portfolio sorts, stocks are ranked each period by their sorting variable and divided into quantile portfolios. Portfolio returns are equal-weighted or value-weighted averages of constituent

stock returns. The long-short factor return is the difference between the extreme portfolios, for instance bottom minus top for asset growth, reflecting the negative premium documented by Cooper, Gulen, and Schill (2008). Key methodological decisions include the number of portfolios, the exchange universe for computing breakpoints, the weighting scheme, and the rebalancing frequency.

The `tidyfinance` package implements this workflow through three functions. First, `breakpoint_options()` configures the sorting parameters: the number of quantile portfolios and which exchanges determine the breakpoints. Second, `compute_portfolio_returns()` takes the sorting data, applies the specified configuration, and returns portfolio-level returns. Third, `compute_long_short_returns()` constructs the hedge portfolio return from the extreme quantiles.

Univariate sorts

`compute_portfolio_returns()` is built on two lower-level functions that are also available for custom workflows. `compute_breakpoints()` computes quantile cutpoints for the sorting variable from a specified exchange universe. `assign_portfolio()` maps each stock-month to a portfolio number based on those cutpoints. Researchers who need finer control, for instance to build overlapping portfolios, apply non-standard cutoffs, or weight within cells differently, can call these functions directly rather than through the high-level wrapper.

In a univariate sort, stocks are ranked on a single characteristic. We illustrate this with asset growth, forming decile portfolios using NYSE breakpoints and value-weighted returns, rebalanced annually in July.

```
options_main <- breakpoint_options(
  n_portfolios = 5,
  breakpoint_exchanges = "NYSE"
)
```

The `breakpoint_options()` call specifies that breakpoints come from NYSE-listed stocks only, a common convention that avoids the influence of small AMEX and NASDAQ firms on portfolio cutoffs. We then pass these options to `compute_portfolio_returns()`.

```
portfolio_returns <- compute_portfolio_returns(
  sorting_data,
  sorting_variables = "asset_growth",
  sorting_method = "univariate",
  rebalancing_month = 7,
  options_main
)
```

```
Returning a complete panel with 55 missing values in factor returns due to
insufficient observations (n() < 0).
```

`compute_portfolio_returns()` returns a long tibble with one row per portfolio-month, containing value-weighted (`ret_excess_vw`), equal-weighted (`ret_excess_ew`), and capped value-weighted (`ret_excess_vw_capped`) excess returns. The long-short factor return is obtained via `compute_long_short_returns()`. The `direction = "bottom_minus_top"` argument means the factor is long low-AG stocks and short high-AG stocks, consistent with the negative premium documented by Cooper, Gulen, and Schill (2008).

```
factor_returns_ag <- portfolio_returns |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table **??** summarises the time series of each quintile across all three weighting schemes, with the long-short portfolio appended as the final row.

Table 4: Time-series averages of annualised monthly excess returns for asset growth quintile portfolios. Portfolios are formed annually in July using NYSE breakpoints, rebalanced annually. Means are annualised (multiplied by 12). The t~statistic tests whether the mean monthly excess return differs from zero. The L/S row is the long-short portfolio (bottom minus top quintile).

Portfolio	VW Mean	VW t-stat	EW Mean	EW t-stat	VWC Mean	VWC t-stat
1	0.093	4.349	0.149	4.816	0.103	4.085
2	0.079	4.343	0.117	5.139	0.095	4.530
3	0.071	3.796	0.108	4.845	0.092	4.425
4	0.073	3.677	0.103	4.297	0.088	3.935
5	0.067	2.667	0.050	1.630	0.056	1.981
L/S	0.026	1.924	0.099	9.116	0.047	4.468

The value-weighted long-short portfolio earns 2.6% per year (t~statistic 1.92), consistent with the negative premium documented by Cooper, Gulen, and Schill (2008). The equal-weighted spread is larger, as expected for characteristics that correlate with size. The monotone decline across quintiles confirms that `tidyfinance` reproduces the literature result from raw WRDS data.

Bivariate sorts

Bivariate (double) sorts control for a second variable, typically firm size (`mktcap_lag`), when forming portfolios. In dependent sorts, stocks are first sorted into groups by the secondary

variable; within each group, they are then sorted on the main characteristic. In independent sorts, both sorts apply unconditionally, and portfolios form at the intersections. The factor return averages across the secondary dimension, isolating the effect of the primary variable after controlling for size. Figure **??** illustrates the three approaches for a single cross-section.

We reuse `options_main` from the univariate sort (five portfolios, NYSE breakpoints) and add a separate options object for the secondary size dimension:

```
options_secondary <- breakpoint_options(
  n_portfolios = 2,
  breakpoint_exchanges = "NYSE"
)

portfolio_returns_biv <- compute_portfolio_returns(
  sorting_data,
  c("asset_growth", "mktcap_lag"),
  sorting_method = "bivariate-dependent",
  rebalancing_month = 7,
  options_main,
  options_secondary
)
```

```
Returning a complete panel with 55 missing values in factor returns due to
insufficient observations (n() < 0).
```

```
factor_returns_biv <- portfolio_returns_biv |>
  compute_long_short_returns(direction = "bottom_minus_top")
```

Table 5: Univariate vs size-controlled long-short returns for asset growth. Annualized value-weighted mean excess return and t-statistic for the univariate sort and the dependent bivariate sort averaging across two size groups. Portfolios use NYSE breakpoints and annual July rebalancing.

Sort	VW Mean	VW t-stat
Univariate	0.026	1.924
Bivariate (size-controlled)	0.036	3.376

The AG premium survives size control, suggesting it captures an investment effect distinct from the size factor. The `sorting_method` argument switches between dependent and independent double sorts, so researchers can inspect both configurations without modifying the pipeline.

Factor library and API

> **Note**
>
> This section introduces the full factor library and API.
>
> - Explain the computation of the entire grid of factor returns across all combinations of methodological choices. Cite WWW and Gruber et al. for the motivation of this comprehensive approach.
> - Emphasise that the library is available for download, so researchers can access results without re-running the computationally intensive portfolio sort pipeline.
> - Show how to download one default time series and how to use `fill_all`.
> - Illustrate Sharpe-ratio variation.
> - Mention that ´py-tidyfinance‘ can be used with identical callss

Specification grid

The examples above represent single points in a high-dimensional space of methodological choices. Table **??** summarises the preprocessing decisions that can be varied when constructing factor returns. The full factor library exhaustively evaluates all valid combinations of these choices for each of the sorting variables listed in Table **??**.

Table 6: Preprocessing choices in the portfolio sort grid and corresponding `download_data()` arguments. Italic values indicate the default. For double sorts, the secondary quantiles and exchange choices apply to the size dimension only; for univariate sorts, those rows are not applicable. FF refers to the lag procedure of Eugene F. Fama and French (1993).

Preprocessing choice	Argument	Options	Description
Size exclusion	`exclude_size`	None; *bottom 20%*	Fraction of the smallest stocks (by NYSE market cap) excluded from the universe.
Exclusion of financials	`exclude_financials`	*No*; Yes	Drops financial-sector stocks (SIC 6000–6999) from the universe.
Exclusion of utilities	`exclude_utilities`	*No*; Yes	Drops utility-sector stocks (SIC 4900–4999) from the universe.
Exclusion of neg. earnings	`exclude_negative_earnings`	*No*; Yes	Drops firms with negative earnings before sorting.

Table 6: Preprocessing choices in the portfolio sort grid and corresponding `download_data()` arguments. Italic values indicate the default. For double sorts, the secondary quantiles and exchange choices apply to the size dimension only; for univariate sorts, those rows are not applicable. FF refers to the lag procedure of Eugene F. Fama and French (1993).

Preprocessing choice	Argument	Options	Description
Sorting variable lag	`sorting_variable_lag`	3; 1; *6m*; FF	Lag applied to the sorting variable before portfolio assignment.
Rebalancing	`rebalancing`	*Monthly*; Annual	How frequently portfolios are reformed.
BP: Quantiles (main)	`breakpoints_main`	3; 5; *10*	Number of quantile groups for the portfolio sort.
Sort method	`sorting_method`	*Single*; Double dependent; Double independent	Single sort or sequential double sort (by size).
BP: Quantiles (secondary)	`breakpoints_secondary`	*Ndep*; 2; 5	Groups for the secondary sort variable; only relevant for double sorts.
BP: Exchanges	`breakpoints_exchanges`	*NYSE*; All	Exchange(s) used to compute quantile breakpoints.
Weighting scheme	`weighting_scheme`	EW; *VW*; truncated VW	Return weighting within portfolios.

We host precomputed factor returns for all specifications on Hugging Face and make them accessible via the `tidyfinance` API. Researchers can retrieve results without re-running the computationally intensive portfolio sort pipeline. An interactive download interface is also available at factors.tidy-finance.org.

The factor library covers 50 sorting variables across seven categories: investment, profitability, valuation, momentum, intangibles, trading frictions, and financing. The selection follows Jensen, Kelly, and Pedersen (2023). Table **??** lists all variables with their long-short direction and primary reference.

The factor library

This sensitivity is not specific to AG. For all 50 sorting variables in the library, construction choices shift Sharpe ratios by enough to reverse the sign of the premium for some factors. Figure **??** shows the distribution of annualized Sharpe ratios for each variable across all 187008 specifications.

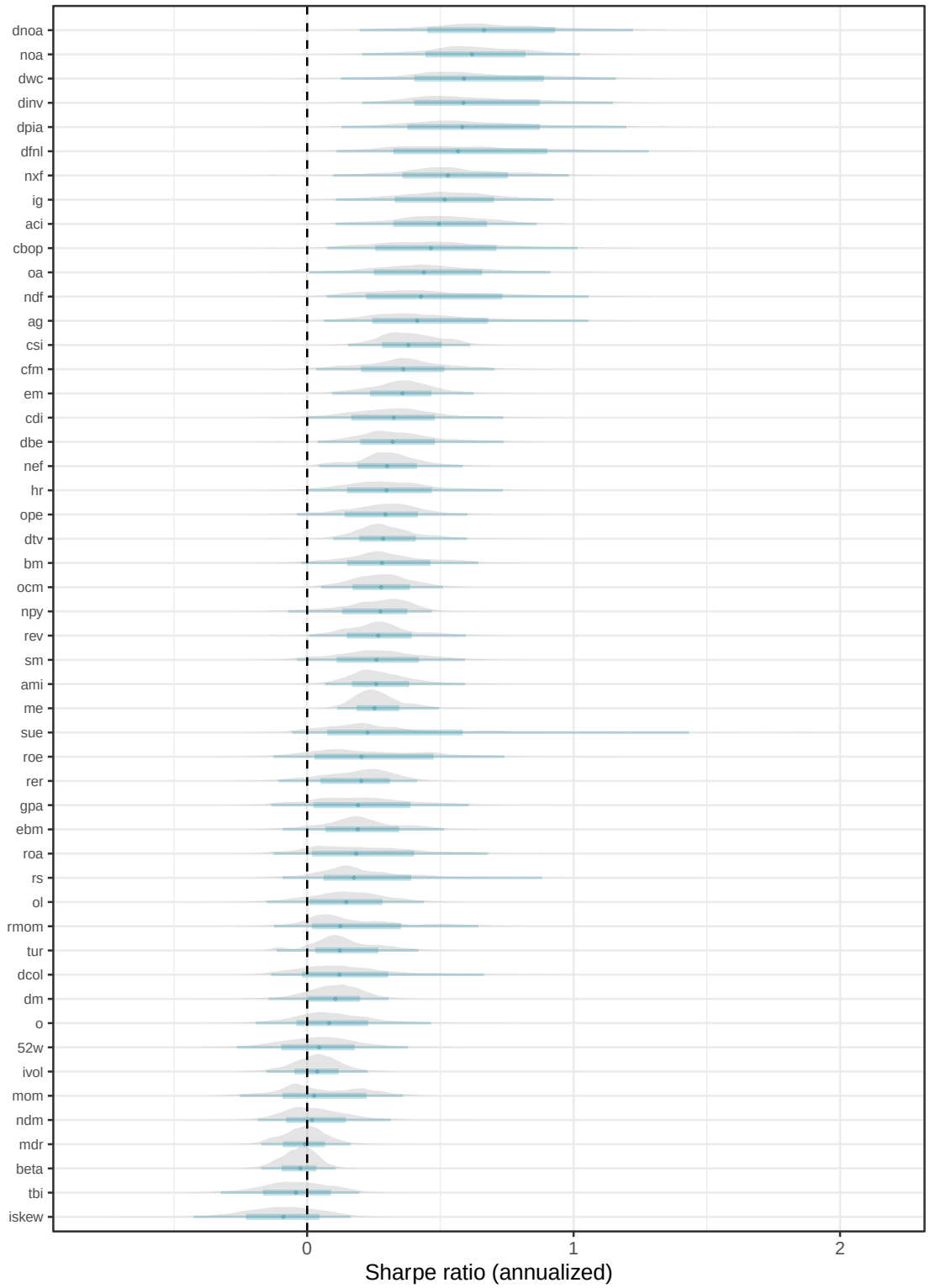


Figure 5: Sharpe ratios across construction choices. Ridge plots show the distribution of annualized Sharpe ratios of long-short return differentials for each sorting variable across all `r .QuartoInlineRender(20row(portfolio_sort_grid))` specifications illustrated in Table **??**, based on the available sample period per factor. Most variables exhibit substantial variation across specifications, underscoring the sensitivity of factor returns to construction choices.